TRƯỜNG ĐẠI HỌC ĐÀ LẠT
KHOA CÔNG NGHỆ THÔNG TIN

BÀI THỰC HÀNH 2

NEXT.JS — NỀN TẢNG & ROUTING

Môn học: Các công nghệ mới trong phát triển phần mềm
Buổi: 2
Thời lượng: 4 tiết (180 phút)

Đà Lạt, 2026

MỤC LỤC

Mục tiêu bài thực hành 1

Yêu cầu đầu vào 1

Quy ước ký hiệu 1

Giới thiệu: Xây dựng website Portfolio/Blog cá nhân 1

Phần 1: Kiến trúc App Router trong Next.JS 2

- **1.1. Nguyên lý file-based routing** 2
- **1.2. Các file đặc biệt trong App Router** 2
- **1.3. Thực hành: Tìm hiểu cấu trúc project hiện tại** 3
- **Bài tập tự làm — Phần 1** 4

Phần 2: Pages & Nested Layouts 4

- **2.1. Tạo layout dùng chung (Navigation & Footer)** 4
- **2.2. Tạo các trang cơ bản** 7
- **2.3. Kiểm tra kết quả** 10
- **2.4. Nested Layouts (Layout lồng nhau)** 11
- **2.5. Commit thay đổi** 12
- **Bài tập tự làm — Phần 2** 12

Phần 3: Dynamic Routes & Route Groups 12

- **3.1. Dynamic Routes với [slug]** 12
- **3.2. Route Groups** 17
- **3.3. Kiểm tra kết quả** 17
- **3.4. Commit thay đổi** 18
- **Bài tập tự làm — Phần 3** 18

Phần 4: Loading State & Error Boundaries 18

- **4.1. Loading State với loading.tsx** 19
- **4.2. Error Boundary với error.tsx** 19
- **4.3. Trang 404 tùy chỉnh với not-found.tsx** 20
- **4.4. Kiểm tra kết quả** 22
- **4.5. Commit thay đổi** 22
- **Bài tập tự làm — Phần 4** 22

Phần 5: Tailwind CSS cơ bản 22

- **5.1. Utility-first CSS là gì?** 23
- **5.2. Các class phổ biến** 23
- **5.3. Responsive Design** 26
- **5.4. Thực hành: Cải thiện trang Dự án với Responsive Grid** 26

5.5. Thực hành: Cải thiện trang chủ 28
5.6. Commit thay đổi 29
Bài tập tự làm — Phần 5 30
Phần 6: Server Components vs Client Components 30
6.1. Server Components (mặc định) 30
6.2. Client Components ("use client") 31
6.3. Thực hành: Tạo component tương tác 31
6.4. Kiểm tra kết quả 34
6.5. Commit thay đổi 34
Bài tập tự làm — Phần 6 34
Phần 7: Tổng kết & Yêu cầu nộp bài 35
Tóm tắt kiến thức 35
Checklist hoàn thành 35
Yêu cầu nộp bài 36
Chuẩn bị cho buổi sau 36
Tài liệu tham khảo 37

Mục tiêu bài thực hành

Sau khi hoàn thành bài thực hành này, sinh viên có khả năng:

1. Hiểu và vận dụng kiến trúc App Router của Next.JS (file-based routing)
2. Tạo được pages, nested layouts, dynamic routes và route groups
3. Xử lý loading state, error boundaries và trang 404 tùy chỉnh
4. Sử dụng Tailwind CSS để xây dựng giao diện responsive
5. Phân biệt và sử dụng đúng Server Components vs Client Components

Yêu cầu đầu vào

Kiến thức nền:

- Hoàn thành Bài thực hành 1 (đã có project Next.JS, Git, Node.js)
- Kiến thức React cơ bản: component, props, state, JSX
- Biết sử dụng terminal và Git workflow cơ bản

Phần cứng & phần mềm:

- Máy tính đã cài đặt Node.js, VS Code, Git (từ Bài TH 1)
- Project `ctk46-lab01` đã khởi tạo và push lên GitHub

Quy ước ký hiệu

Ký hiệu	Ý nghĩa
	Mẹo hữu ích
	Lưu ý quan trọng
	Bài tập tự làm
	Giải thích thêm
	Kết quả mong đợi

Giới thiệu: Xây dựng website Portfolio/Blog cá nhân

Trong bài thực hành này, chúng ta sẽ xây dựng một **website portfolio/blog cá nhân** hoàn chỉnh trên nền tảng project `ctk46-lab01` đã tạo ở Bài TH 1. Website sẽ bao gồm các trang:

Trang	URL	Mô tả
Trang chủ	`/`	Giới thiệu ngắn gọn bản thân
Giới thiệu	`/about`	Thông tin chi tiết về bản thân
Blog	`/blog`	Danh sách bài viết
Chi tiết bài viết	`/blog/bai-viet-1`	Nội dung chi tiết từng bài viết (dynamic route)
Dự án	`/projects`	Danh sách dự án đã thực hiện
Liên hệ	`/contact`	Thông tin liên hệ

Qua việc xây dựng website này, bạn sẽ thực hành tất cả các khái niệm routing quan trọng trong Next.JS.

Phần 1: Kiến trúc App Router trong Next.JS

Thời lượng: *Thời lượng: ~20 phút*

App Router là hệ thống routing mặc định của Next.JS (từ phiên bản 13 trở lên). Thay vì cấu hình routes trong file riêng, Next.JS sử dụng **file-based routing** — cấu trúc thư mục quyết định cấu trúc URL của ứng dụng.

1.1. Nguyên lý file-based routing

Trong Next.JS App Router, mỗi thư mục trong `src/app/` tương ứng với một đoạn (segment) trong URL. Để tạo một trang, bạn chỉ cần tạo file `page.tsx` trong thư mục tương ứng.

Quy tắc cơ bản:

```
Cấu trúc thư mục                    URL tương ứng
────────────────────                ─────────────
src/app/page.tsx                  →  /
src/app/about/page.tsx            →  /about
src/app/blog/page.tsx             →  /blog
src/app/blog/[slug]/page.tsx      →  /blog/bai-viet-1, /blog/bai-viet-2, ...
src/app/contact/page.tsx          →  /contact
```

Giải thích: Chỉ file có tên `page.tsx` mới tạo ra route có thể truy cập được. Các file khác trong thư mục (components, utils, styles) không ảnh hưởng đến routing.

1.2. Các file đặc biệt trong App Router

Next.JS quy định một số tên file đặc biệt, mỗi file có vai trò riêng trong hệ thống routing:

File	Vai trò	Phạm vi
`page.tsx`	Nội dung trang — **bắt buộc** để tạo route	Route hiện tại
`layout.tsx`	Layout bao bọc trang (navbar, sidebar, footer)	Route hiện tại + tất cả route con
`loading.tsx`	Giao diện hiển thị khi đang tải trang	Route hiện tại + tất cả route con
`error.tsx`	Giao diện hiển thị khi có lỗi	Route hiện tại + tất cả route con
`not-found.tsx`	Trang 404 tùy chỉnh	Route hiện tại
`template.tsx`	Tương tự layout nhưng tạo instance mới khi chuyển trang	Route hiện tại + tất cả route con

Giải thích: Sự khác biệt giữa `layout.tsx` và `template.tsx`: Layout giữ nguyên state khi chuyển giữa các trang con (không re-render). Template tạo lại từ đầu mỗi khi chuyển trang (re-mount). Trong hầu hết trường hợp, bạn sẽ dùng `layout.tsx`.

Thứ tự render (từ ngoài vào trong):

```
layout.tsx
  └── template.tsx (nếu có)
        └── loading.tsx (khi đang tải)
        └── error.tsx (khi có lỗi)
        └── not-found.tsx (khi không tìm thấy)
        └── page.tsx (nội dung chính)
```

1.3. Thực hành: Tìm hiểu cấu trúc project hiện tại

Mở project `ctk46-lab01` đã tạo ở Bài TH 1:

```bash
cd ctk46-lab01
code .
```

Xem cấu trúc thư mục `src/app/`:

```bash
ls -la src/app/
```

Kết quả mong đợi: Kết quả mong đợi: bạn sẽ thấy các file đã có từ Bài TH 1:

```
src/app/
├── favicon.ico
├── globals.css
├── layout.tsx      ← Root layout (layout gốc, bao bọc toàn bộ app)
└── page.tsx        ← Trang chủ (/)
```

Mẹo: File `layout.tsx` ở `src/app/` là **root layout** — layout gốc bao bọc toàn bộ ứng dụng. Đây là file duy nhất **bắt buộc** phải có thẻ `<html>` và `<body>`.

Bài tập tự làm — Phần 1

1. Mở project `ctk46-lab01` và xác định các file đặc biệt hiện có trong `src/app/`
2. Đọc nội dung file `layout.tsx` và `page.tsx`, xác định vai trò của từng file
3. Thử tạo thư mục `src/app/test/` và file `page.tsx` đơn giản bên trong, sau đó truy cập `http://localhost:3000/test` để xác nhận routing hoạt động
4. Sau khi xác nhận, xóa thư mục `test/` (đây chỉ là thử nghiệm)

Phần 2: Pages & Nested Layouts

Thời lượng: *Thời lượng: ~35 phút*

Trong phần này, chúng ta sẽ tạo nhiều trang cho website portfolio/blog và xây dựng layout dùng chung với navigation bar và footer.

2.1. Tạo layout dùng chung (Navigation & Footer)

Trước khi tạo các trang, ta cần xây dựng layout chung gồm thanh navigation (navbar) và footer để hiển thị trên mọi trang.

Bước 1: Tạo component Navbar

Tạo thư mục components và file Navbar:

```bash
mkdir -p src/components
```

Tạo file `src/components/navbar.tsx`:

```tsx
```

```
import Link from "next/link";

export default function Navbar() {
  return (
    <nav className="bg-white shadow-sm border-b">
      <div className="max-w-5xl mx-auto px-4 py-4">
        <div className="flex items-center justify-between">
          <Link href="/" className="text-xl font-bold text-blue-600">
            Portfolio
          </Link>
          <div className="flex gap-6">
            <Link
              href="/"
              className="text-gray-600 hover:text-blue-600 transition-colors"
            >
              Trang chủ
            </Link>
            <Link
              href="/about"
              className="text-gray-600 hover:text-blue-600 transition-colors"
            >
              Giới thiệu
            </Link>
            <Link
              href="/blog"
              className="text-gray-600 hover:text-blue-600 transition-colors"
            >
              Blog
            </Link>
            <Link
              href="/projects"
              className="text-gray-600 hover:text-blue-600 transition-colors"
            >
              Dự án
            </Link>
            <Link
              href="/contact"
              className="text-gray-600 hover:text-blue-600 transition-colors"
            >
              Liên hệ
            </Link>
          </div>
        </div>
      </div>
    </nav>
  );
}
```

Giải thích: Giải thích:

- `import Link from "next/link"` — Component `Link` của Next.JS dùng để chuyển trang. Khác với thẻ `<a>` thông thường, `Link` thực hiện **client-side navigation** — chỉ tải phần nội dung thay đổi, không tải lại toàn bộ trang.
- `className="..."` — Các class Tailwind CSS (sẽ học chi tiết ở Phần 5).

Lưu ý: Quan trọng: Luôn sử dụng `<Link>` thay vì `<a>` cho các liên kết nội bộ trong ứng dụng Next.JS. `<Link>` giúp chuyển trang nhanh hơn và giữ nguyên state của ứng dụng.

Bước 2: Tạo component Footer

Tạo file `src/components/footer.tsx`:

```tsx
export default function Footer() {
  return (
    <footer className="bg-gray-50 border-t mt-auto">
      <div className="max-w-5xl mx-auto px-4 py-6 text-center text-gray-500 text-
sm">
        <p>
          © 2025 — Nguyễn Văn A | CTK46 — Các công nghệ mới trong PTPM
        </p>
      </div>
    </footer>
  );
}
```

Bước 3: Cập nhật Root Layout

Mở file `src/app/layout.tsx` và cập nhật để tích hợp Navbar và Footer:

```tsx
import type { Metadata } from "next";
import "./globals.css";
import Navbar from "@/components/navbar";
import Footer from "@/components/footer";

export const metadata: Metadata = {
  title: "Portfolio — Nguyễn Văn A",
  description: "Website portfolio cá nhân — CTK46",
};

export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html lang="vi">
      <body className="min-h-screen flex flex-col">
        <Navbar />
        <main className="flex-1">{children}</main>
        <Footer />
      </body>
    </html>
  );
}
```

Giải thích: Giải thích:

- `@/components/navbar` — Ký hiệu `@/` là alias trỏ đến thư mục `src/`. Đây là cấu hình mặc định khi tạo project Next.JS.
- `children` — Nội dung của trang hiện tại, sẽ được hiển thị giữa Navbar và Footer.
- `min-h-screen flex flex-col` — Đảm bảo layout chiếm toàn bộ chiều cao màn hình, footer luôn ở dưới cùng.
- `flex-1` — Phần nội dung chính (`main`) chiếm toàn bộ không gian còn lại.

2.2. Tạo các trang cơ bản

Trang chủ (/)

Cập nhật file `src/app/page.tsx`:

```tsx
export default function HomePage() {
  return (
    <div className="max-w-5xl mx-auto px-4 py-12">
      <div className="text-center">
        <h1 className="text-4xl font-bold mb-4">
          Xin chào! Tôi là Nguyễn Văn A
        </h1>
        <p className="text-xl text-gray-600 mb-2">
          Sinh viên Công nghệ Thông tin – Đại học Đà Lạt
        </p>
        <p className="text-lg text-gray-500">
          Đam mê phát triển web và các công nghệ mới
        </p>
      </div>
    </div>
  );
}
```

Trang Giới thiệu (/about)

Tạo thư mục và file:

```bash
mkdir -p src/app/about
```

Tạo file `src/app/about/page.tsx`:

```tsx
export default function AboutPage() {
```

```
  return (
    <div className="max-w-5xl mx-auto px-4 py-12">
      <h1 className="text-3xl font-bold mb-6">Giới thiệu</h1>

      <div className="space-y-4 text-gray-700">
        <p>
          Xin chào! Tôi là <strong>Nguyễn Văn A</strong>, sinh viên năm 4
          ngành Công nghệ Thông tin tại Đại học Đà Lạt.
        </p>

        <h2 className="text-2xl font-semibold mt-8 mb-4">Kỹ năng</h2>
        <ul className="list-disc list-inside space-y-2">
          <li>JavaScript / TypeScript</li>
          <li>React & Next.JS</li>
          <li>Tailwind CSS</li>
          <li>Git & GitHub</li>
          <li>SQL & PostgreSQL</li>
        </ul>

        <h2 className="text-2xl font-semibold mt-8 mb-4">Học vấn</h2>
        <div className="bg-gray-50 rounded-lg p-4">
          <p className="font-medium">Đại học Đà Lạt</p>
          <p className="text-gray-500">Cử nhân Công nghệ Thông tin (2021 –
2025)</p>
        </div>
      </div>
    </div>
  );
}
```

Trang Liên hệ (/contact)

```bash
mkdir -p src/app/contact
```

Tạo file `src/app/contact/page.tsx`:

```tsx
export default function ContactPage() {
  return (
    <div className="max-w-5xl mx-auto px-4 py-12">
      <h1 className="text-3xl font-bold mb-6">Liên hệ</h1>

      <div className="space-y-4 text-gray-700">
        <div className="bg-gray-50 rounded-lg p-6 space-y-3">
          <p>
            <strong>Email:</strong>{" "}
            <a
              href="mailto:nguyenvana@sv.dlu.edu.vn"
              className="text-blue-600 hover:underline"
            >
              nguyenvana@sv.dlu.edu.vn
            </a>
          </p>
```

```
          <p>
            <strong>GitHub:</strong>{" "}
            <a
              href="https://github.com/nguyenvana"
              target="_blank"
              rel="noopener noreferrer"
              className="text-blue-600 hover:underline"
            >
              github.com/nguyenvana
            </a>
          </p>
          <p>
            <strong>Địa chỉ:</strong> Đại học Đà Lạt, 01 Phù Đổng Thiên
            Vương, Đà Lạt
          </p>
        </div>
      </div>
    </div>
  );
}
```

Trang Dự án (/projects)

```bash
mkdir -p src/app/projects
```

Tạo file `src/app/projects/page.tsx`:

```tsx
const projects = [
  {
    title: "Website Portfolio",
    description: "Website cá nhân xây dựng bằng Next.JS và Tailwind CSS",
    tech: ["Next.JS", "Tailwind CSS", "TypeScript"],
  },
  {
    title: "Ứng dụng Quản lý Công việc",
    description: "Ứng dụng Todo App với React và Local Storage",
    tech: ["React", "CSS Modules", "JavaScript"],
  },
  {
    title: "API RESTful",
    description: "API quản lý sản phẩm với Node.js và Express",
    tech: ["Node.js", "Express", "MongoDB"],
  },
];

export default function ProjectsPage() {
  return (
    <div className="max-w-5xl mx-auto px-4 py-12">
      <h1 className="text-3xl font-bold mb-6">Dự án</h1>

      <div className="grid gap-6">
        {projects.map((project, index) => (
```

```
          <div
            key={index}
            className="border rounded-lg p-6 hover:shadow-md transition-shadow"
          >
            <h2 className="text-xl font-semibold mb-2">{project.title}</h2>
            <p className="text-gray-600 mb-4">{project.description}</p>
            <div className="flex gap-2">
              {project.tech.map((t) => (
                <span
                  key={t}
                  className="bg-blue-100 text-blue-700 text-sm px-3 py-1 rounded-
full"
                >
                  {t}
                </span>
              ))}
            </div>
          </div>
        ))}
      </div>
    </div>
  );
}
```

Giải thích: Giải thích: Ở đây chúng ta sử dụng mảng `projects` làm **dữ liệu giả (mock data)** và dùng `map()` để render danh sách. Ở các buổi sau, dữ liệu sẽ được lấy từ database (Supabase).

2.3. Kiểm tra kết quả

Khởi động development server:

```bash
npm run dev
```

Truy cập lần lượt các URL sau và kiểm tra:

URL	Kết quả mong đợi
`http://localhost:3000`	Trang chủ với navbar + footer
`http://localhost:3000/about`	Trang giới thiệu
`http://localhost:3000/contact`	Trang liên hệ
`http://localhost:3000/projects`	Trang dự án

Kết quả mong đợi: Mọi trang đều hiển thị Navbar ở trên và Footer ở dưới. Khi nhấn vào các liên kết trên Navbar, trang chuyển mượt mà mà không tải lại toàn bộ (nhờ `<Link>` component).

Mẹo: Để ý rằng khi chuyển trang, **chỉ phần nội dung giữa Navbar và Footer thay đổi** — Navbar và Footer không bị render lại. Đây chính là cơ chế hoạt động của `layout.tsx`.

2.4. Nested Layouts (Layout lồng nhau)

Ngoài root layout, bạn có thể tạo layout riêng cho từng section. Ví dụ, tạo layout riêng cho phần blog với sidebar hiển thị danh mục.

Tạo thư mục blog (sẽ dùng ở các phần tiếp theo):

```bash
mkdir -p src/app/blog
```

Tạo file `src/app/blog/layout.tsx`:

```tsx
export default function BlogLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <div className="max-w-5xl mx-auto px-4 py-12">
      <div className="flex gap-8">
        {/* Nội dung chính */}
        <div className="flex-1">{children}</div>

        {/* Sidebar */}
        <aside className="w-64 shrink-0">
          <div className="bg-gray-50 rounded-lg p-4">
            <h3 className="font-semibold mb-3">Danh mục</h3>
            <ul className="space-y-2 text-sm text-gray-600">
              <li className="hover:text-blue-600 cursor-pointer">
                Công nghệ
              </li>
              <li className="hover:text-blue-600 cursor-pointer">
                Học tập
              </li>
              <li className="hover:text-blue-600 cursor-pointer">
                Dự án cá nhân
              </li>
              <li className="hover:text-blue-600 cursor-pointer">
                Cuộc sống
              </li>
            </ul>
          </div>
        </aside>
      </div>
    </div>
  );
}
```

Giải thích: Layout lồng nhau hoạt động như thế nào?

Khi truy cập `/blog`, Next.JS render theo thứ tự:

```
Root Layout (src/app/layout.tsx)          ← Navbar + Footer
  └── Blog Layout (src/app/blog/layout.tsx)  ← Sidebar
        └── Blog Page (src/app/blog/page.tsx)    ← Nội dung
```

Điều này nghĩa là trang blog sẽ có Navbar (từ root layout) + Sidebar (từ blog layout) + nội dung chính.

2.5. Commit thay đổi

Dừng dev server (`Ctrl+C`) và commit:

```bash
git add .
git commit -m "feat: thêm layout với navbar, footer và các trang cơ bản"
git push origin main
```

Bài tập tự làm — Phần 2

1. Cập nhật nội dung trang chủ, giới thiệu, liên hệ với **thông tin cá nhân thật** của bạn
2. Thêm ít nhất **3 dự án** vào trang `/projects` (có thể là bài tập, đồ án đã làm)
3. Chỉnh sửa Navbar: thay tên "Portfolio" bằng tên thật của bạn
4. Thử tạo thêm một trang mới `/skills` liệt kê các kỹ năng lập trình
5. Thử sửa Footer: thêm link đến GitHub và email cá nhân

Phần 3: Dynamic Routes & Route Groups

Thời lượng: *Thời lượng: ~35 phút*

Trong phần trước, chúng ta tạo các trang với URL cố định (`/about`, `/contact`). Nhưng với blog hay danh sách sản phẩm, mỗi bài viết/sản phẩm cần một URL riêng — ví dụ `/blog/bai-viet-1`, `/blog/bai-viet-2`. Đây là lúc **dynamic routes** phát huy tác dụng.

3.1. Dynamic Routes với [slug]

Dynamic routes sử dụng tên thư mục trong ngoặc vuông `[param]` để tạo routes có phần URL thay đổi.

Bước 1: Tạo dữ liệu giả cho blog

Tạo file `src/data/posts.ts` để lưu dữ liệu bài viết:

```bash
mkdir -p src/data
```

```ts
export interface Post {
  slug: string;
  title: string;
  excerpt: string;
  content: string;
  date: string;
  category: string;
}

export const posts: Post[] = [
  {
    slug: "gioi-thieu-nextjs",
    title: "Giới thiệu Next.JS — Framework React phổ biến nhất",
    excerpt:
      "Tìm hiểu tại sao Next.JS là lựa chọn hàng đầu cho phát triển web hiện
đại.",
    content: `Next.JS là một React framework mạnh mẽ được phát triển bởi Vercel.
    Nó cung cấp nhiều tính năng quan trọng như Server-Side Rendering (SSR),
    Static Site Generation (SSG), và App Router.

    Một số ưu điểm nổi bật của Next.JS:
    - Routing tự động dựa trên cấu trúc thư mục
    - Hỗ trợ Server Components và Client Components
    - Tối ưu hóa hình ảnh, font, và scripts tự động
    - API Routes tích hợp
    - Hỗ trợ TypeScript sẵn có`,
    date: "2025-01-15",
    category: "Công nghệ",
  },
  {
    slug: "hoc-tailwind-css",
    title: "Tailwind CSS — Cách tiếp cận mới cho CSS",
    excerpt:
      "Khám phá phương pháp utility-first CSS và tại sao nó thay đổi cách viết
CSS.",
    content: `Tailwind CSS là một utility-first CSS framework, nghĩa là thay vì
viết
    CSS tùy chỉnh, bạn sử dụng các class tiện ích có sẵn để xây dựng giao diện.

    Ví dụ, thay vì viết:
    .card { padding: 16px; border-radius: 8px; box-shadow: 0 2px 4px
rgba(0,0,0,0.1); }
```

```
    Bạn viết trực tiếp trong HTML:
    <div class="p-4 rounded-lg shadow-md">...</div>

    Ưu điểm:
    - Không cần đặt tên class
    - Không cần chuyển qua lại giữa file HTML và CSS
    - File CSS cuối cùng rất nhỏ (chỉ chứa class đã dùng)`,
    date: "2025-01-20",
    category: "Công nghệ",
  },
  {
    slug: "kinh-nghiem-hoc-lap-trinh",
    title: "Chia sẻ kinh nghiệm tự học lập trình hiệu quả",
    excerpt:
      "Những bài học rút ra sau 3 năm tự học lập trình ở đại học.",
    content: `Sau 3 năm học tập và thực hành lập trình, tôi rút ra một số kinh
    nghiệm quan trọng:

    1. Thực hành nhiều hơn đọc lý thuyết
    Lập trình là kỹ năng thực hành. Đọc sách và xem video chỉ chiếm 30%,
    70% còn lại là viết code.

    2. Xây dựng dự án thực tế
    Không gì tốt hơn việc xây dựng một sản phẩm thực tế để học.
    Hãy bắt đầu từ những dự án nhỏ và tăng dần độ phức tạp.

    3. Tham gia cộng đồng
    Tham gia các cộng đồng lập trình để học hỏi và chia sẻ kinh nghiệm.`,
    date: "2025-02-01",
    category: "Học tập",
  },
];

export function getPostBySlug(slug: string): Post | undefined {
  return posts.find((post) => post.slug === slug);
}
```

Bước 2: Tạo trang danh sách blog

Tạo file `src/app/blog/page.tsx`:

```tsx
import Link from "next/link";
import { posts } from "@/data/posts";

export default function BlogPage() {
  return (
    <div>
      <h1 className="text-3xl font-bold mb-6">Blog</h1>

      <div className="space-y-6">
        {posts.map((post) => (
          <article
            key={post.slug}
            className="border rounded-lg p-6 hover:shadow-md transition-shadow"
```

```
          >
            <div className="flex items-center gap-3 mb-2">
              <span className="bg-blue-100 text-blue-700 text-xs px-2 py-1
rounded">
                {post.category}
              </span>
              <span className="text-sm text-gray-400">{post.date}</span>
            </div>

            <Link href={`/blog/${post.slug}`}>
              <h2 className="text-xl font-semibold mb-2 hover:text-blue-600
transition-colors">
                {post.title}
              </h2>
            </Link>

            <p className="text-gray-600">{post.excerpt}</p>

            <Link
              href={`/blog/${post.slug}`}
              className="inline-block mt-3 text-blue-600 text-sm hover:underline"
            >
              Đọc thêm →
            </Link>
          </article>
        ))}
      </div>
    </div>
  );
}
```

Bước 3: Tạo dynamic route cho trang chi tiết bài viết

Tạo thư mục dynamic route:

```bash
mkdir -p src/app/blog/\[slug\]
```

Lưu ý: Trong terminal, cần escape ký tự `[` và `]` bằng dấu `\`. Trong VS Code, bạn có thể tạo thư mục trực tiếp bằng cách nhấn chuột phải → New Folder → nhập `[slug]`.

Tạo file `src/app/blog/[slug]/page.tsx`:

```tsx
import Link from "next/link";
import { notFound } from "next/navigation";
import { getPostBySlug, posts } from "@/data/posts";

interface BlogPostPageProps {
  params: Promise<{ slug: string }>;
}
```

```
export default async function BlogPostPage({ params }: BlogPostPageProps) {
  const { slug } = await params;
  const post = getPostBySlug(slug);

  if (!post) {
    notFound();
  }

  return (
    <div>
      <Link
        href="/blog"
        className="text-blue-600 hover:underline text-sm mb-6 inline-block"
      >
        ← Quay lại danh sách
      </Link>

      <article>
        <div className="flex items-center gap-3 mb-4">
          <span className="bg-blue-100 text-blue-700 text-xs px-2 py-1 rounded">
            {post.category}
          </span>
          <span className="text-sm text-gray-400">{post.date}</span>
        </div>

        <h1 className="text-3xl font-bold mb-6">{post.title}</h1>

        <div className="prose max-w-none text-gray-700 whitespace-pre-line">
          {post.content}
        </div>
      </article>
    </div>
  );
}

export async function generateStaticParams() {
  return posts.map((post) => ({
    slug: post.slug,
  }));
}
```

Giải thích: Giải thích chi tiết:

- `[slug]` — Thư mục có tên trong ngoặc vuông tạo **dynamic segment**. Giá trị `slug` sẽ là phần URL tương ứng (ví dụ: truy cập `/blog/gioi-thieu-nextjs` thì `slug = "gioi-thieu-nextjs"`).
- `params: Promise<{ slug: string }>` — Next.JS truyền giá trị dynamic segment vào props của page component. Trong phiên bản App Router mới, `params` là Promise nên cần `await`.

- `notFound()` — Hàm từ `next/navigation`, khi gọi sẽ hiển thị trang 404. Ở đây dùng khi bài viết không tồn tại.
- `generateStaticParams()` — Hàm cho Next.JS biết trước danh sách các `slug` hợp lệ. Giúp Next.JS tạo sẵn (pre-render) các trang tĩnh, tăng tốc độ tải.

3.2. Route Groups

Route Groups cho phép **tổ chức thư mục** mà **không ảnh hưởng đến URL**. Tên thư mục được đặt trong ngoặc tròn `(group-name)`.

Ví dụ thực tế: Giả sử bạn muốn tách ứng dụng thành 2 phần — phần **marketing** (trang chủ, giới thiệu) và phần **dashboard** (quản lý nội dung) — với layout khác nhau cho mỗi phần.

```
src/app/
├── (marketing)/
│   ├── layout.tsx          ← Layout cho phần marketing (có navbar)
│   ├── page.tsx            ← / (trang chủ)
│   └── about/
│       └── page.tsx        ← /about
├── (dashboard)/
│   ├── layout.tsx          ← Layout cho phần dashboard (có sidebar)
│   └── admin/
│       └── page.tsx        ← /admin
└── layout.tsx              ← Root layout
```

URL kết quả:

Thư mục	URL
`(marketing)/page.tsx`	`/`
`(marketing)/about/page.tsx`	`/about`
`(dashboard)/admin/page.tsx`	`/admin`

Mẹo: Tên trong ngoặc tròn `(marketing)`, `(dashboard)` **không xuất hiện trong URL**. Chúng chỉ dùng để tổ chức code.

Lưu ý: Lưu ý: Chúng ta sẽ không cấu trúc lại project theo Route Groups trong bài lab này vì project còn nhỏ. Hãy ghi nhớ kỹ thuật này — nó rất hữu ích khi project lớn hơn (từ Buổi 5 trở đi).

3.3. Kiểm tra kết quả

Khởi động dev server và kiểm tra:

```bash
npm run dev
```

URL	Kết quả mong đợi
`http://localhost:3000/blog`	Danh sách 3 bài viết, có sidebar danh mục bên phải
`http://localhost:3000/blog/gioi-thieu-nextjs`	Nội dung chi tiết bài viết "Giới thiệu Next.JS"
`http://localhost:3000/blog/hoc-tailwind-css`	Nội dung chi tiết bài viết "Tailwind CSS"
`http://localhost:3000/blog/khong-ton-tai`	Trang 404 (bài viết không tồn tại)

Kết quả mong đợi: Trang blog hiển thị danh sách bài viết với sidebar. Nhấn vào bài viết sẽ chuyển đến trang chi tiết. Truy cập slug không tồn tại sẽ hiển thị trang 404.

3.4. Commit thay đổi

```bash
git add .
git commit -m "feat: thêm blog với dynamic routes và mock data"
git push origin main
```

Bài tập tự làm — Phần 3

1. Thêm ít nhất **2 bài viết** nữa vào file `src/data/posts.ts` (nội dung tự chọn)
2. Thêm trường `author` vào interface `Post` và hiển thị trên trang chi tiết
3. Tạo trang `/projects/[id]` hiển thị chi tiết từng dự án (tương tự blog)
4. Thử tạo catch-all route `src/app/blog/[...slug]/page.tsx` thay vì `[slug]` — quan sát sự khác biệt khi truy cập `/blog/a/b/c`
5. Nghiên cứu thêm: đọc tài liệu chính thức về Route Groups tại https://nextjs.org/docs/app/building-your-application/routing/route-groups (https://nextjs.org/docs/app/building-your-application/routing/route-groups)

Phần 4: Loading State & Error Boundaries

Thời lượng: *Thời lượng: ~25 phút*

Trong ứng dụng web thực tế, dữ liệu không phải lúc nào cũng sẵn sàng ngay lập tức — có thể mất vài giây để tải từ API hoặc database. Và đôi khi, lỗi sẽ xảy ra. Next.JS cung cấp cơ chế xử lý cả hai trường hợp này một cách gọn gàng thông qua các file đặc biệt.

4.1. Loading State với loading.tsx

File `loading.tsx` được hiển thị tự động khi Next.JS đang tải nội dung trang. Điều này tạo trải nghiệm tốt hơn cho người dùng — thay vì nhìn màn hình trắng, họ sẽ thấy giao diện loading.

Tạo loading cho toàn bộ blog

Tạo file `src/app/blog/loading.tsx`:

```tsx
export default function BlogLoading() {
  return (
    <div>
      <div className="h-8 w-48 bg-gray-200 rounded animate-pulse mb-6" />

      <div className="space-y-6">
        {[1, 2, 3].map((i) => (
          <div key={i} className="border rounded-lg p-6">
            <div className="flex gap-3 mb-3">
              <div className="h-5 w-20 bg-gray-200 rounded animate-pulse" />
              <div className="h-5 w-24 bg-gray-200 rounded animate-pulse" />
            </div>
            <div className="h-6 w-3/4 bg-gray-200 rounded animate-pulse mb-2" />
            <div className="h-4 w-full bg-gray-200 rounded animate-pulse" />
            <div className="h-4 w-2/3 bg-gray-200 rounded animate-pulse mt-1" />
          </div>
        ))}
      </div>
    </div>
  );
}
```

Giải thích: Giải thích: Đây là kỹ thuật **skeleton loading** — tạo các hình chữ nhật xám với animation `animate-pulse` (nhấp nháy) để mô phỏng bố cục nội dung sẽ hiển thị. Skeleton loading tạo cảm giác trang đang tải nhanh hơn so với spinner xoay.

Mẹo: Hiện tại với mock data, loading sẽ hiển thị rất nhanh (gần như không thấy). Khi tích hợp database thực tế ở Buổi 4-5, bạn sẽ thấy rõ tác dụng của loading state.

4.2. Error Boundary với error.tsx

File `error.tsx` được hiển thị khi có lỗi runtime xảy ra trong trang hoặc các component con. Nó ngăn lỗi "lan ra" toàn bộ ứng dụng — chỉ phần bị lỗi hiển thị thông báo lỗi, phần còn lại (navbar, footer) vẫn hoạt động bình thường.

Tạo file `src/app/blog/error.tsx`:

```tsx
"use client";

export default function BlogError({
  error,
  reset,
}: {
  error: Error & { digest?: string };
  reset: () => void;
}) {
  return (
    <div className="text-center py-12">
      <h2 className="text-2xl font-bold text-red-600 mb-4">
        Đã xảy ra lỗi!
      </h2>
      <p className="text-gray-600 mb-6">
        {error.message || "Không thể tải nội dung blog. Vui lòng thử lại."}
      </p>
      <button
        onClick={() => reset()}
        className="bg-blue-600 text-white px-6 py-2 rounded-lg hover:bg-blue-700
transition-colors"
      >
        Thử lại
      </button>
    </div>
  );
}
```

Lưu ý: Bắt buộc: File `error.tsx` phải là **Client Component** (có `"use client"` ở dòng đầu tiên). Đây là yêu cầu của Next.JS vì error boundary sử dụng React Error Boundary API — chỉ hoạt động ở phía client.

Giải thích: Giải thích props:

- `error` — Đối tượng lỗi JavaScript, chứa thông tin chi tiết về lỗi
- `reset()` — Hàm thử render lại nội dung. Khi người dùng nhấn "Thử lại", Next.JS sẽ cố gắng render lại trang — hữu ích khi lỗi là tạm thời (ví dụ: mất kết nối mạng)

4.3. Trang 404 tùy chỉnh với not-found.tsx

Trang 404 cho blog

Tạo file `src/app/blog/[slug]/not-found.tsx`:

```tsx
import Link from "next/link";

export default function PostNotFound() {
  return (
    <div className="text-center py-12">
      <h2 className="text-2xl font-bold mb-4">Bài viết không tồn tại</h2>
      <p className="text-gray-600 mb-6">
        Bài viết bạn đang tìm kiếm không có trong hệ thống.
      </p>
      <Link
        href="/blog"
        className="bg-blue-600 text-white px-6 py-2 rounded-lg hover:bg-blue-700
transition-colors inline-block"
      >
        Quay lại Blog
      </Link>
    </div>
  );
}
```

Trang 404 toàn cục

Tạo file `src/app/not-found.tsx` — áp dụng cho toàn bộ ứng dụng:

```tsx
import Link from "next/link";

export default function NotFound() {
  return (
    <div className="max-w-5xl mx-auto px-4 py-24 text-center">
      <h1 className="text-6xl font-bold text-gray-300 mb-4">404</h1>
      <h2 className="text-2xl font-bold mb-4">Trang không tồn tại</h2>
      <p className="text-gray-600 mb-8">
        Xin lỗi, trang bạn đang tìm kiếm không có trên website này.
      </p>
      <Link
        href="/"
        className="bg-blue-600 text-white px-6 py-2 rounded-lg hover:bg-blue-700
transition-colors inline-block"
      >
        Về trang chủ
      </Link>
    </div>
  );
}
```

Giải thích: Cách hoạt động: Khi gọi `notFound()` trong `src/app/blog/[slug]/page.tsx`, Next.JS sẽ tìm file `not-found.tsx` gần nhất — nếu có trong `[slug]/` thì dùng, nếu không thì dùng file ở `src/app/not-found.tsx`.

4.4. Kiểm tra kết quả

```bash
npm run dev
```

URL	Kết quả mong đợi
`http://localhost:3000/blog`	Danh sách bài viết (loading skeleton hiển thị ngắn)
`http://localhost:3000/blog/khong-ton-tai`	Trang "Bài viết không tồn tại"
`http://localhost:3000/trang-khong-co`	Trang 404 toàn cục

Kết quả mong đợi: Khi truy cập URL không hợp lệ, trang 404 tùy chỉnh hiển thị thay vì trang lỗi mặc định.

4.5. Commit thay đổi

```bash
git add .
git commit -m "feat: thêm loading state, error boundary và trang 404"
git push origin main
```

Bài tập tự làm — Phần 4

1. Tạo file `loading.tsx` cho trang chi tiết bài viết (`src/app/blog/[slug]/loading.tsx`) với skeleton phù hợp
2. Tạo file `error.tsx` ở cấp root (`src/app/error.tsx`) cho toàn bộ ứng dụng
3. Tùy chỉnh trang 404 toàn cục: thêm hình ảnh minh họa hoặc animation
4. Thử cố ý tạo lỗi trong code (ví dụ: gọi method trên `undefined`) và quan sát error boundary hoạt động

Phần 5: Tailwind CSS cơ bản

Thời lượng: *Thời lượng: ~35 phút*

Trong các phần trước, chúng ta đã sử dụng Tailwind CSS mà chưa giải thích chi tiết. Phần này sẽ giúp bạn hiểu rõ cách Tailwind CSS hoạt động và sử dụng thành thạo các class phổ biến nhất.

5.1. Utility-first CSS là gì?

Tailwind CSS theo triết lý **utility-first** — thay vì viết CSS riêng cho mỗi component, bạn sử dụng các class tiện ích (utility classes) nhỏ, mỗi class thực hiện đúng 1 việc.

So sánh CSS truyền thống vs Tailwind:

CSS truyền thống:

```Css
/* styles.css */
.card {
  padding: 24px;
  border-radius: 8px;
  border: 1px solid #e5e7eb;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.1);
}

.card-title {
  font-size: 20px;
  font-weight: 600;
  margin-bottom: 8px;
}
```

```Html
<div class="card">
  <h2 class="card-title">Tiêu đề</h2>
</div>
```

Tailwind CSS:

```Html
<div class="p-6 rounded-lg border border-gray-200 shadow-sm">
  <h2 class="text-xl font-semibold mb-2">Tiêu đề</h2>
</div>
```

Mẹo: Ưu điểm: Không cần đặt tên class, không cần chuyển qua lại giữa file HTML và CSS, không lo CSS bị xung đột. File CSS cuối cùng chỉ chứa các class đã sử dụng (rất nhỏ gọn).

5.2. Các class phổ biến

Spacing (khoảng cách)

Class	CSS tương ứng	Mô tả
`p-4`	`padding: 16px`	Padding 4 đơn vị (1 đơn vị = 4px)
`px-4`	`padding-left: 16px;` `padding-right: 16px`	Padding ngang
`py-4`	`padding-top: 16px;` `padding-bottom: 16px`	Padding dọc
`pt-4`	`padding-top: 16px`	Padding trên
`m-4`	`margin: 16px`	Margin 4 đơn vị
`mb-4`	`margin-bottom: 16px`	Margin dưới
`space-y-4`	Khoảng cách dọc giữa các phần tử con	16px giữa mỗi child
`gap-4`	`gap: 16px`	Khoảng cách trong flex/grid

Giải thích: Quy tắc số: Mỗi đơn vị = 4px. `p-1` = 4px, `p-2` = 8px, `p-4` = 16px, `p-8` = 32px.

Typography (chữ)

Class	CSS tương ứng	Mô tả
`text-sm`	`font-size: 14px`	Chữ nhỏ
`text-base`	`font-size: 16px`	Chữ bình thường (mặc định)
`text-lg`	`font-size: 18px`	Chữ lớn
`text-xl`	`font-size: 20px`	Chữ rất lớn
`text-2xl`	`font-size: 24px`	Tiêu đề nhỏ
`text-3xl`	`font-size: 30px`	Tiêu đề vừa
`text-4xl`	`font-size: 36px`	Tiêu đề lớn
`font-bold`	`font-weight: 700`	In đậm
`font-semibold`	`font-weight: 600`	Hơi đậm
`text-center`	`text-align: center`	Căn giữa
`text-gray-600`	`color: #4B5563`	Màu xám
`text-blue-600`	`color: #2563EB`	Màu xanh

Layout (bố cục)

Class	CSS tương ứng	Mô tả
`flex`	`display: flex`	Bố cục flex
`flex-col`	`flex-direction: column`	Flex theo chiều dọc
`items-center`	`align-items: center`	Căn giữa theo trục phụ
`justify-between`	`justify-content: space-between`	Dàn đều hai đầu
`justify-center`	`justify-content: center`	Căn giữa theo trục chính
`grid`	`display: grid`	Bố cục grid
`grid-cols-2`	`grid-template-columns: repeat(2, 1fr)`	Grid 2 cột
`grid-cols-3`	`grid-template-columns: repeat(3, 1fr)`	Grid 3 cột
`max-w-5xl`	`max-width: 1024px`	Chiều rộng tối đa
`mx-auto`	`margin-left: auto; margin-right: auto`	Căn giữa ngang
`w-full`	`width: 100%`	Chiều rộng 100%

Decoration (trang trí)

Class	CSS tương ứng	Mô tả
`bg-white`	`background-color: #fff`	Nền trắng
`bg-gray-50`	`background-color: #F9FAFB`	Nền xám rất nhạt
`bg-blue-600`	`background-color: #2563EB`	Nền xanh
`rounded`	`border-radius: 4px`	Bo góc nhẹ
`rounded-lg`	`border-radius: 8px`	Bo góc vừa
`rounded-full`	`border-radius: 9999px`	Bo tròn hoàn toàn
`border`	`border: 1px solid`	Viền mặc định
`shadow-sm`	Đổ bóng nhẹ	Hiệu ứng đổ bóng
`shadow-md`	Đổ bóng vừa	Hiệu ứng đổ bóng

Interaction (tương tác)

Class	Mô tả
`hover:bg-blue-700`	Đổi nền khi hover
`hover:text-blue-600`	Đổi màu chữ khi hover

`hover:underline`	Gạch chân khi hover
`hover:shadow-md`	Thêm đổ bóng khi hover
`transition-colors`	Animation mượt khi đổi màu
`transition-shadow`	Animation mượt khi đổi bóng
`cursor-pointer`	Đổi con trỏ thành bàn tay

5.3. Responsive Design

Tailwind CSS sử dụng **mobile-first breakpoints** — style mặc định áp dụng cho tất cả kích thước, sau đó ghi đè cho màn hình lớn hơn.

Breakpoints:

Prefix	Kích thước tối thiểu	Thiết bị
(không có)	0px	Mobile (mặc định)
`sm:`	640px	Tablet nhỏ
`md:`	768px	Tablet
`lg:`	1024px	Laptop
`xl:`	1280px	Desktop
`2xl:`	1536px	Màn hình lớn

Ví dụ responsive grid:

```tsx
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
  {/* 1 cột trên mobile, 2 cột trên tablet, 3 cột trên desktop */}
</div>
```

5.4. Thực hành: Cải thiện trang Dự án với Responsive Grid

Cập nhật file `src/app/projects/page.tsx` để sử dụng responsive grid:

```tsx
const projects = [
  {
    title: "Website Portfolio",
    description: "Website cá nhân xây dựng bằng Next.JS và Tailwind CSS",
    tech: ["Next.JS", "Tailwind CSS", "TypeScript"],
    status: "Đang phát triển",
  },
  {
    title: "Ứng dụng Quản lý Công việc",
    description: "Ứng dụng Todo App với React và Local Storage",
```

```
    tech: ["React", "CSS Modules", "JavaScript"],
    status: "Hoàn thành",
  },
  {
    title: "API RESTful",
    description: "API quản lý sản phẩm với Node.js và Express",
    tech: ["Node.js", "Express", "MongoDB"],
    status: "Hoàn thành",
  },
  {
    title: "Chat Realtime",
    description: "Ứng dụng chat realtime với Socket.IO",
    tech: ["React", "Socket.IO", "Node.js"],
    status: "Đang phát triển",
  },
];

export default function ProjectsPage() {
  return (
    <div className="max-w-5xl mx-auto px-4 py-12">
      <h1 className="text-3xl font-bold mb-6">Dự án</h1>

      <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
        {projects.map((project, index) => (
          <div
            key={index}
            className="border rounded-lg p-6 hover:shadow-md transition-shadow
flex flex-col"
          >
            <div className="flex items-center justify-between mb-2">
              <h2 className="text-xl font-semibold">{project.title}</h2>
              <span
                className={`text-xs px-2 py-1 rounded-full ${
                  project.status === "Hoàn thành"
                    ? "bg-green-100 text-green-700"
                    : "bg-yellow-100 text-yellow-700"
                }`}
              >
                {project.status}
              </span>
            </div>

            <p className="text-gray-600 mb-4 flex-1">{project.description}</p>

            <div className="flex flex-wrap gap-2">
              {project.tech.map((t) => (
                <span
                  key={t}
                  className="bg-blue-100 text-blue-700 text-sm px-3 py-1 rounded-
full"
                >
                  {t}
                </span>
              ))}
            </div>
          </div>
        ))}
      </div>
```

```
    </div>
  );
}
```

Giải thích: Điểm mới:

- `grid grid-cols-1 md:grid-cols-2` — 1 cột trên mobile, 2 cột từ tablet trở lên
- `flex flex-col` + `flex-1` — Đảm bảo các card có chiều cao bằng nhau
- `flex-wrap` — Cho phép các tag tech xuống dòng khi không đủ chỗ
- Conditional class: `project.status === "Hoàn thành" ? "bg-green-..." : "bg-yellow-..."` — Đổi màu badge theo trạng thái

5.5. Thực hành: Cải thiện trang chủ

Cập nhật `src/app/page.tsx` với thiết kế đẹp hơn:

```tsx
import Link from "next/link";

export default function HomePage() {
  return (
    <div className="max-w-5xl mx-auto px-4 py-16">
      {/* Hero section */}
      <div className="text-center mb-16">
        <div className="w-24 h-24 bg-blue-100 rounded-full mx-auto mb-6 flex
items-center justify-center">
          <span className="text-4xl">A</span>
        </div>

        <h1 className="text-4xl md:text-5xl font-bold mb-4">
          Xin chào! Tôi là{" "}
          <span className="text-blue-600">Nguyễn Văn A</span>
        </h1>
        <p className="text-xl text-gray-600 mb-8 max-w-2xl mx-auto">
          Sinh viên Công nghệ Thông tin tại Đại học Đà Lạt. Đam mê phát triển
          web và khám phá các công nghệ mới.
        </p>

        <div className="flex justify-center gap-4">
          <Link
            href="/projects"
            className="bg-blue-600 text-white px-6 py-3 rounded-lg hover:bg-blue-
700 transition-colors"
          >
            Xem dự án
          </Link>
          <Link
            href="/contact"
            className="border border-gray-300 px-6 py-3 rounded-lg hover:bg-gray-
50 transition-colors"
          >
```

```
            Liên hệ
          </Link>
        </div>
      </div>

      {/* Skills section */}
      <div className="mb-16">
        <h2 className="text-2xl font-bold text-center mb-8">Kỹ năng</h2>
        <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
          {[
            "JavaScript",
            "TypeScript",
            "React",
            "Next.JS",
            "Tailwind CSS",
            "Node.js",
            "Git",
            "SQL",
          ].map((skill) => (
            <div
              key={skill}
              className="bg-gray-50 rounded-lg p-4 text-center hover:bg-blue-50
hover:text-blue-600 transition-colors"
            >
              {skill}
            </div>
          ))}
        </div>
      </div>

      {/* CTA section */}
      <div className="bg-blue-50 rounded-2xl p-8 text-center">
        <h2 className="text-2xl font-bold mb-3">Đọc Blog của tôi</h2>
        <p className="text-gray-600 mb-4">
          Chia sẻ kiến thức và kinh nghiệm về lập trình, công nghệ
        </p>
        <Link
          href="/blog"
          className="text-blue-600 font-semibold hover:underline"
        >
          Xem blog →
        </Link>
      </div>
    </div>
  );
}
```

Kết quả mong đợi: Kiểm tra trên `http://localhost:3000` — trang chủ giờ đã có hero section, danh sách kỹ năng dạng grid, và phần call-to-action. Thử thu nhỏ cửa sổ trình duyệt để thấy responsive design hoạt động.

5.6. Commit thay đổi

```bash
git add .
git commit -m "feat: cải thiện giao diện với Tailwind CSS responsive"
git push origin main
```

Bài tập tự làm — Phần 5

1. Thay thế thông tin cá nhân trên trang chủ bằng thông tin thật của bạn
2. Cập nhật danh sách kỹ năng phản ánh đúng kỹ năng thực tế của bạn
3. Thử thay đổi color scheme: thay tất cả `blue` thành `emerald`, `violet`, hoặc `orange`
4. Làm cho navbar responsive: trên mobile (kích thước nhỏ), ẩn các link và hiển thị nút menu (hamburger icon)
5. Thêm **dark mode**: sử dụng class `dark:` để thay đổi màu sắc (ví dụ: `dark:bg-gray-900 dark:text-white`). Tham khảo: https://tailwindcss.com/docs/dark-mode (https://tailwindcss.com/docs/dark-mode)
6. Tra cứu thêm class Tailwind CSS tại https://tailwindcss.com/docs (https://tailwindcss.com/docs) — thử áp dụng ít nhất 5 class mới chưa dùng trong bài

Phần 6: Server Components vs Client Components

Thời lượng: *Thời lượng: ~20 phút*

Một trong những khái niệm quan trọng nhất trong Next.JS App Router là sự phân biệt giữa **Server Components** và **Client Components**. Hiểu rõ khi nào dùng loại nào sẽ giúp bạn xây dựng ứng dụng hiệu quả hơn.

6.1. Server Components (mặc định)

Trong Next.JS App Router, **tất cả component mặc định là Server Component** — chúng được render trên server, không có JavaScript phía client.

Ưu điểm:

- Không gửi JavaScript đến trình duyệt → trang tải nhanh hơn
- Có thể truy cập trực tiếp database, file system, API nội bộ
- Bảo mật hơn (code không lộ ở phía client)

Hạn chế:

- Không dùng được React hooks: `useState`, `useEffect`, `useRef`,...
- Không dùng được event handlers: `onClick`, `onChange`,...
- Không truy cập được Browser APIs: `window`, `document`, `localStorage`,...

Giải thích: Tất cả các component chúng ta đã viết ở các phần trước (Navbar, Footer, các page) đều là **Server Components**. Chúng hiển thị nội dung tĩnh, không cần tương tác → hoàn toàn phù hợp.

6.2. Client Components ("use client")

Khi component cần **tương tác với người dùng** (nhấn nút, nhập liệu, thay đổi state), bạn cần chuyển sang Client Component bằng cách thêm `"use client"` ở dòng đầu tiên.

Khi nào cần `"use client"`:

Tính năng	Cần Client Component?
Hiển thị text, hình ảnh tĩnh	Không — Server Component
Fetch data từ API	Không — Server Component
`onClick`, `onChange`, `onSubmit`	**Có**
`useState`, `useEffect`	**Có**
`useRouter()` hook	**Có**
`window`, `document`, `localStorage`	**Có**
Import thư viện chỉ hoạt động trên trình duyệt	**Có**

6.3. Thực hành: Tạo component tương tác

Counter (Bộ đếm)

Tạo file `src/components/counter.tsx`:

```tsx
"use client";

import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
```

```
    <div className="flex items-center gap-4">
      <button
        onClick={() => setCount(count - 1)}
        className="bg-gray-200 hover:bg-gray-300 w-10 h-10 rounded-lg text-xl
font-bold transition-colors"
      >
        -
      </button>
      <span className="text-2xl font-bold w-12 text-center">{count}</span>
      <button
        onClick={() => setCount(count + 1)}
        className="bg-blue-600 hover:bg-blue-700 text-white w-10 h-10 rounded-lg
text-xl font-bold transition-colors"
      >
        +
      </button>
    </div>
  );
}
```

LikeButton (Nút thích)

Tạo file `src/components/like-button.tsx`:

```tsx
"use client";

import { useState } from "react";

export default function LikeButton() {
  const [liked, setLiked] = useState(false);
  const [count, setCount] = useState(0);

  const handleLike = () => {
    if (liked) {
      setCount(count - 1);
    } else {
      setCount(count + 1);
    }
    setLiked(!liked);
  };

  return (
    <button
      onClick={handleLike}
      className={`flex items-center gap-2 px-4 py-2 rounded-lg border transition-
colors ${
        liked
          ? "bg-red-50 border-red-200 text-red-600"
          : "bg-white border-gray-200 text-gray-600 hover:bg-gray-50"
      }`}
    >
      <span>{liked ? "❤️" : "🤍"}</span>
      <span>{count} lượt thích</span>
    </button>
```

```
  );
}
```

Sử dụng Client Components trong Server Component

Thêm Counter vào trang chủ và LikeButton vào trang blog. Cập nhật `src/app/blog/[slug]/page.tsx` — thêm LikeButton vào cuối bài viết:

```tsx
import Link from "next/link";
import { notFound } from "next/navigation";
import { getPostBySlug, posts } from "@/data/posts";
import LikeButton from "@/components/like-button";

interface BlogPostPageProps {
  params: Promise<{ slug: string }>;
}

export default async function BlogPostPage({ params }: BlogPostPageProps) {
  const { slug } = await params;
  const post = getPostBySlug(slug);

  if (!post) {
    notFound();
  }

  return (
    <div>
      <Link
        href="/blog"
        className="text-blue-600 hover:underline text-sm mb-6 inline-block"
      >
        ← Quay lại danh sách
      </Link>

      <article>
        <div className="flex items-center gap-3 mb-4">
          <span className="bg-blue-100 text-blue-700 text-xs px-2 py-1 rounded">
            {post.category}
          </span>
          <span className="text-sm text-gray-400">{post.date}</span>
        </div>

        <h1 className="text-3xl font-bold mb-6">{post.title}</h1>

        <div className="prose max-w-none text-gray-700 whitespace-pre-line mb-8">
          {post.content}
        </div>

        <div className="border-t pt-6">
          <LikeButton />
        </div>
      </article>
    </div>
```

```
  );
}

export async function generateStaticParams() {
  return posts.map((post) => ({
    slug: post.slug,
  }));
}
```

Giải thích: Mô hình kết hợp Server + Client Component:

```
BlogPostPage (Server Component)      ← Fetch data, render HTML
  └── <article>...</article>          ← Nội dung tĩnh (server)
  └── <LikeButton />                  ← Client Component (tương tác)
```

Mẹo: Best practice: Giữ phần lớn component là Server Component. Chỉ thêm `"use client"` cho những component nhỏ, cần tương tác. Tránh đặt `"use client"` cho component cha lớn — thay vào đó, tách phần tương tác thành component con riêng.

6.4. Kiểm tra kết quả

```bash
npm run dev
```

URL	Kiểm tra
`http://localhost:3000/blog/gioi-thieu-nextjs`	Cuối bài viết có nút Like, nhấn thay đổi trạng thái

Kết quả mong đợi: Nút Like hoạt động: nhấn đổi từ sang , số lượt thích tăng/giảm. Phần nội dung bài viết vẫn là Server Component (render trên server), chỉ nút Like là Client Component (tương tác trên client).

6.5. Commit thay đổi

```bash
git add .
git commit -m "feat: thêm Client Components tương tác (Counter, LikeButton)"
git push origin main
```

Bài tập tự làm — Phần 6

1. Thêm Counter component vào trang chủ (import và sử dụng trong `page.tsx`)

2. Tạo component `ThemeToggle` với `"use client"`: nút chuyển đổi giữa sáng/tối (chỉ cần thay đổi text và icon, chưa cần thực sự đổi theme)
3. Tạo component `CopyButton` — khi nhấn, copy một đoạn text vào clipboard và hiển thị "Đã copy!"
4. Thử bỏ `"use client"` khỏi file `counter.tsx` và chạy lại — quan sát lỗi xảy ra. Đọc thông báo lỗi và giải thích tại sao
5. Nghiên cứu thêm tại https://nextjs.org/docs/app/building-your-application/rendering/client-components (https://nextjs.org/docs/app/building-your-application/rendering/client-components)

Phần 7: Tổng kết & Yêu cầu nộp bài

Thời lượng: *Thời lượng: ~10 phút*

Tóm tắt kiến thức

Trong bài thực hành này, bạn đã học và thực hành:

Chủ đề	Kiến thức chính
App Router	File-based routing, các file đặc biệt (page, layout, loading, error, not-found)
Pages & Layouts	Tạo pages, root layout, nested layouts, Link component
Dynamic Routes	`[slug]` dynamic segments, `generateStaticParams`, route groups `(group)`
Loading & Error	`loading.tsx` skeleton, `error.tsx` boundary, `not-found.tsx` trang 404
Tailwind CSS	Utility-first, spacing, typography, layout, colors, responsive breakpoints
Server vs Client	Server Components (mặc định), Client Components (`"use client"`), khi nào dùng

Checklist hoàn thành

Trước khi nộp bài, hãy đảm bảo bạn đã hoàn thành tất cả các mục sau:

☐ Tạo layout dùng chung với Navbar và Footer

- ☐ Tạo ít nhất 5 trang: trang chủ, giới thiệu, blog, dự án, liên hệ
- ☐ Tạo dynamic route cho trang chi tiết bài viết (`/blog/[slug]`)
- ☐ Tạo file `loading.tsx` cho blog
- ☐ Tạo file `error.tsx` cho blog
- ☐ Tạo trang 404 tùy chỉnh (toàn cục và cho blog)
- ☐ Sử dụng Tailwind CSS responsive (ít nhất 1 responsive grid)
- ☐ Tạo ít nhất 1 Client Component tương tác (`"use client"`)
- ☐ Cập nhật thông tin cá nhân thật trên các trang
- ☐ Commit với message đúng chuẩn Conventional Commits
- ☐ Push toàn bộ code lên GitHub

Yêu cầu nộp bài

Nội dung nộp:

1. **Link GitHub repository** chứa project đã hoàn thành
2. Repository phải đáp ứng:
   - Ít nhất **5 trang** với routing đúng
   - Ít nhất **1 dynamic route** hoạt động
   - Có **loading state** và **error boundary**
   - Sử dụng **Tailwind CSS responsive**
   - Có ít nhất **1 Client Component** tương tác
   - Commit message đúng chuẩn Conventional Commits

Hình thức nộp: Gửi link repository qua hệ thống quản lý bài tập của lớp

Chuẩn bị cho buổi sau

Buổi 3 sẽ học về **Next.JS — Data Fetching, API Routes & UI**. Để chuẩn bị tốt, hãy:

1. Đọc trước tài liệu Data Fetching: [https://nextjs.org/docs/app/building-your-application/data-fetching (https://nextjs.org/docs/app/building-your-application/data-fetching)](https://nextjs.org/docs/app/building-your-application/data-fetching)

2. Tìm hiểu về Server Actions: https://nextjs.org/docs/app/building-your-application/data-fetching/server-actions-and-mutations (https://nextjs.org/docs/app/building-your-application/data-fetching/server-actions-and-mutations)
3. Xem qua shadcn/ui: https://ui.shadcn.com (https://ui.shadcn.com)
4. Ôn lại kiến thức HTTP methods: GET, POST, PUT, DELETE

Tài liệu tham khảo

- Next.JS App Router: https://nextjs.org/docs/app (https://nextjs.org/docs/app)
- Next.JS Routing: https://nextjs.org/docs/app/building-your-application/routing (https://nextjs.org/docs/app/building-your-application/routing)
- Next.JS Dynamic Routes: https://nextjs.org/docs/app/building-your-application/routing/dynamic-routes (https://nextjs.org/docs/app/building-your-application/routing/dynamic-routes)
- Next.JS Loading UI: https://nextjs.org/docs/app/building-your-application/routing/loading-ui-and-streaming (https://nextjs.org/docs/app/building-your-application/routing/loading-ui-and-streaming)
- Next.JS Error Handling: https://nextjs.org/docs/app/building-your-application/routing/error-handling (https://nextjs.org/docs/app/building-your-application/routing/error-handling)
- Next.JS Server & Client Components: https://nextjs.org/docs/app/building-your-application/rendering/server-components (https://nextjs.org/docs/app/building-your-application/rendering/server-components)
- Tailwind CSS: https://tailwindcss.com/docs (https://tailwindcss.com/docs)
- Tailwind CSS Responsive Design: https://tailwindcss.com/docs/responsive-design (https://tailwindcss.com/docs/responsive-design)